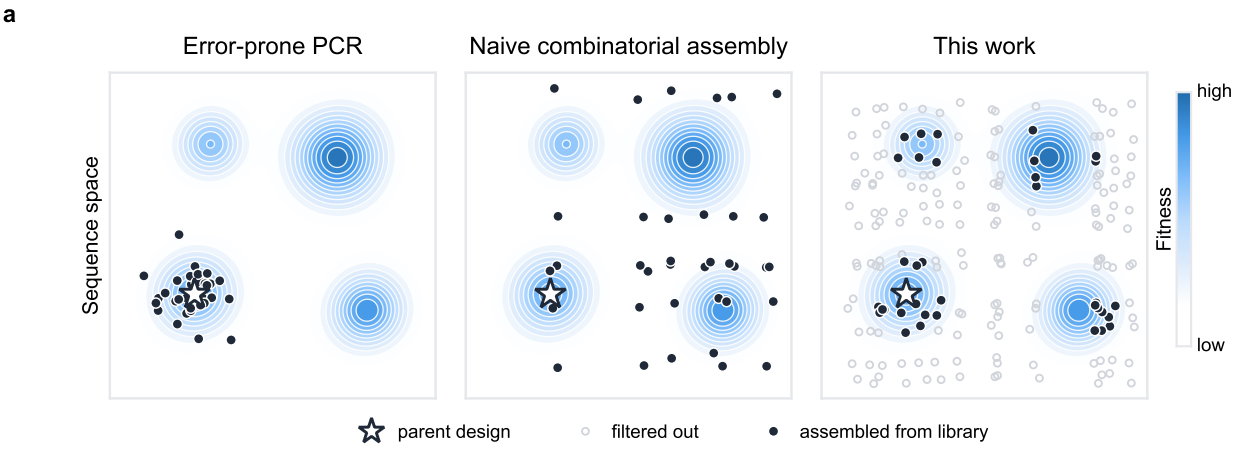
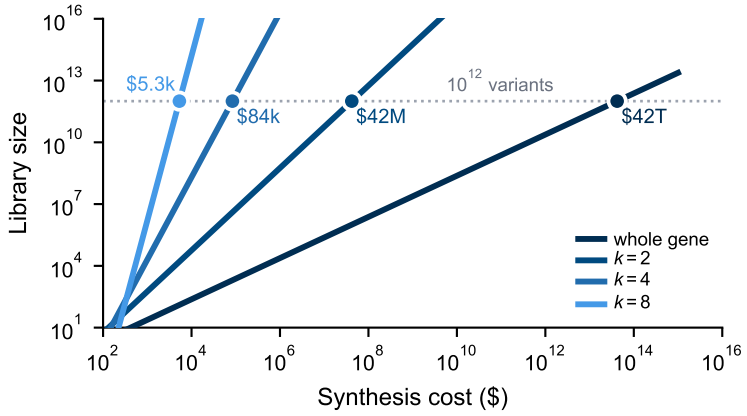
Design of high fidelity combinatorial libraries

November 5, 2026

Abstract

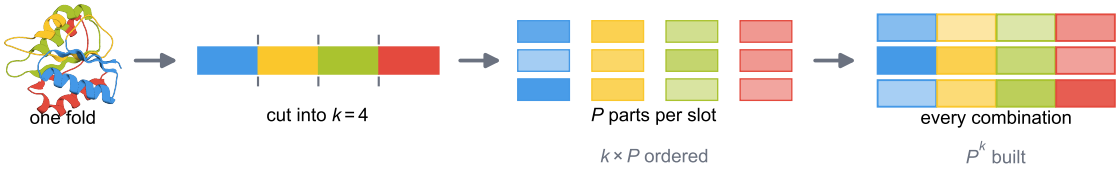
Recent tools for de novo design have revolutionized the way we propose novel proteins and the types of challenges we are able to tackle. While these tools have dramatically increased the hit rate of functional designs, zero-shot generations often require further optimization to enhance the desired properties. We present a novel method for generating high fidelity diverse combinatorial libraries. Traditional library generation methods rely on error prone PCR or natural fragment shuffling to generate diversity. Such approaches do not take into account the filtering of designs through structure prediction models or other desired oracles. We demonstrate the ability to post-train inverse folding models to design fragments with filter pass rates of 93.4% and libraries which can easily scale beyond 10^{12} individually addressable variants. Our post-training framework enables control over the search space by centering the library around a reference sequence with tunable mutation distance. Once designed, we demonstrate how individual variants are assembled with simple molecular biology techniques (golden gate assembly). Our cell-free assay platform enables us to screen enzymes at the scale of 10^3 measurements per day, each with replicates. The libraries generated are easily much larger than most traditional and modern high throughput assays are able to screen, so we demonstrate how the library can be sub sampled smartly to optimize enzymes. For a de novo designed PETase we demonstrate our platform finds a variant with 250x increase in catalytic activity.





c

panel (c) reserved



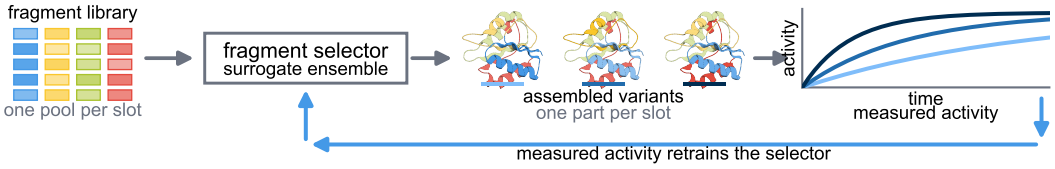
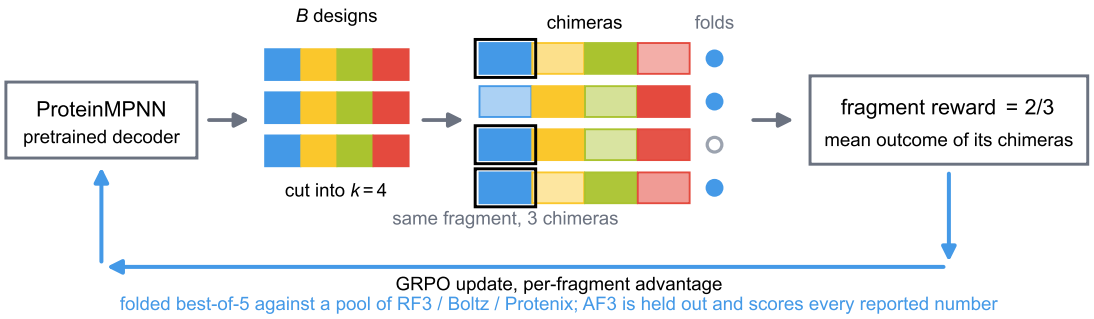


Figure 1: **A combinatorial library, and the loop around it.** [PLACEHOLDER — the prose is yours. Kept short so the float fits; the facts below are the ones a reader cannot get from the picture.] **(a)** One toy landscape searched three ways with the same number of variants built and screened ($N = 36$). Illustrative: no axis is measured. **(b)** eBlocks are billed at a 300-bp minimum, which binds from $k = 2$ on, so past that point cutting finer stops buying shorter DNA and only buys more pieces of it. 10^{12} variants costs \$42T as whole genes, \$42M at $k = 2$, \$84k at $k = 4$, \$5.3k at $k = 8$. **(c)** [Reserved — to be filled in.] **(d)** What makes the library combinatorial. One fold is cut at $k - 1$ boundaries; each slot is then an independent pool of P parts; and any one part per slot assembles a full-length design. Ordering $k \times P$ fragments therefore builds P^k sequences, which is the exponent panel (b) prices. Parts per slot are drawn as three to keep the columns legible. **(e)** The experimental loop. The arrow returns to the *selector*, not the library: parts are synthesised once and each round changes only which of them get combined.

[PLACEHOLDER — the prose is yours. The numbers below are from `paper/figures/panel_cost.py`, which prints them; do not retype them by hand.]

Post-training costs a fixed number of folds. A run is 200 steps, and `build_chimeras` emits rotations × batch = 64 chimeras per step, each folded best-of-5: $4 \times 16 \times 5 = 320$ folds per step, 64 000 in total. The bill does not grow with k — cutting a design finer does not fold more of it — so one figure covers the whole sweep. The alternative is to fold your way to a usable library: generate with stock ProteinMPNN, fold every candidate, and keep what passes. At vanilla pass rate p, obtaining S members that fold costs $5S/p$ folds, which is linear in S, where post-training is a constant 64 000 *however large the library is* — the enrichment lives in the policy, not in the members, so anything drawn from a trained policy inherits the higher pass rate without being folded at all. Below a few thousand members brute force is the cheaper option, and the break-even depends on how good the untrained model already was: 3300 members at M0255's 25.8 %, 6300 at M0315's 49.6 %, 7700 at M0097's 59.8 %. That range is the whole point. A combinatorial library is not a few thousand members: $k = 4$ with 100 parts per slot is 10^8, where filtering by folding would cost $\sim 8 \times 10^8$ folds — four orders of magnitude more than training once — and at 10^{12} the comparison stops being a comparison.

a


B designs
chimeras
folds
ProteinMPNN
pretrained decoder
cut into k = 4
same fragment, 3 chimeras
fragment reward = 2/3
mean outcome of its chimeras
GRPO update, per-fragment advantage
folded best-of-5 against a pool of RF3 / Boltz / Protenix; AF3 is held out and scores every reported number

b

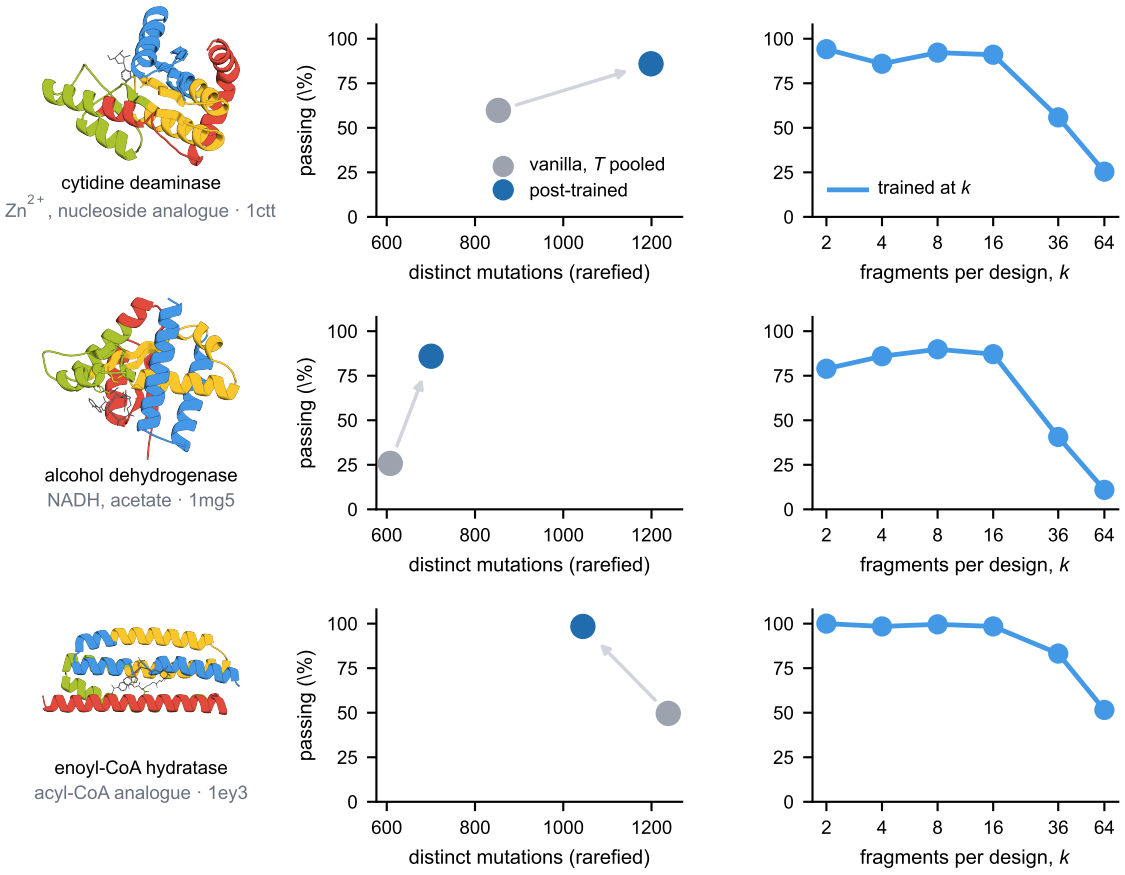

cytidine deaminase
Zn2+, nucleoside analogue · 1ctt
alcohol dehydrogenase
NADH, acetate · 1mg5
enoyl-CoA hydratase
acyl-CoA analogue · 1ey3
passing (%)
distinct mutations (rarefied)
vanilla, T pooled
post-trained
fragments per design, k
trained at k

Figure 2: **Designing high-fidelity combinatorial libraries.** [PLACEHOLDER — the prose is yours.] **(a)** How post-training works here. ProteinMPNN is pretrained and emits whole sequences, so nothing in it says which fragments *recombine*; the method manufactures that signal. A batch is cut into k slots, chimeras are built by swapping slots between parents, each chimera is folded, and every fragment is given the mean outcome of the chimeras it appeared in — here one fragment occurs in three, two of which fold, so its reward is 2/3. That per-fragment reward, not the parent's score, drives the GRPO update. Folding during training is best-of-5 against a *pool* of RF3, Boltz and Protenix under a round robin; AlphaFold3 is held out of training entirely and scores every number in (b)–(d). One row per backbone throughout. **(b)** One row per backbone, read left to right: the target, what post-training buys it, and how far that holds as the library is cut finer. *Left:* the three targets, coloured by their four fragments, with the motif RFdiffusion2 holds fixed drawn as sticks and the ligand in grey. The cut lines run *through* the active site: under the same equal-count split, M0097's motif falls on three different fragments (A26, A59/A61, A93) and M0255's and M0315's each fall on two. Every recombination below is therefore reassembling an active site from parts contributed by different designs. *Middle:* what post-training buys, measured on recombinants — decoded from MPNN, cut at $k = 4$, shuffled across parents, reassembled, folded, never on parents. Arrows run from vanilla to post-trained. Success rises on all three, by +26.2, +60.2 and +48.8 points. The vanilla arm is not a single temperature: its fragments come from parents sampled at $T = 0.1$–1.0 and then filtered to those that pass, which skews the pool toward low T, so it is the filtered union of the whole sweep rather than any one setting. Distinct mutations are counted over *passing* recombinants only and rarefied to a matched survivor count, so a wider spread is genuinely wider coverage and not just a larger sample; two backbones move right as well as up, and M0315 trades 193 mutations for its 48.8 points. *Right:* a policy trained at each k and recombined at that same k, with every cell run for a matched 200 steps so training budget cannot covary with k. Retention is flat through $k = 16$ — 11-residue fragments — and falls away below that. The control — one $k = 4$ policy re-cut at every k, with no training at the finer granularity — is in Fig. S1. Caveats that must survive into the text: every cell is single-seeded against a documented 21-point run-to-run swing; M0255 at $k = 36$ and $k = 64$ was still climbing at step 200, so those are lower bounds.

Supplementary figures

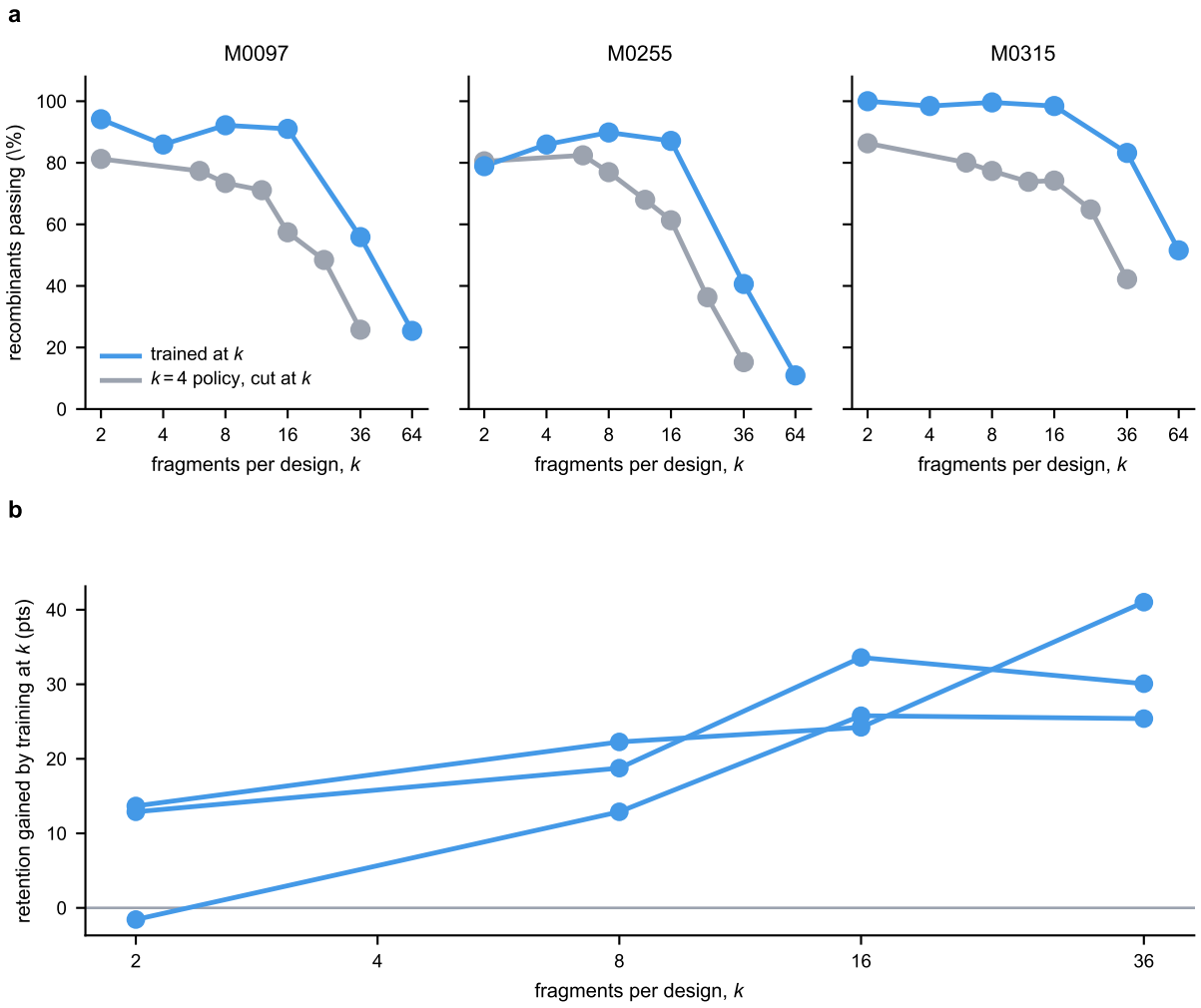

a
M0097
M0255
M0315
recombinants passing (%)
100
80
60
40
20
0
trained at k
k = 4 policy, cut at k
2
4
8
16
36
64
fragments per design, k
b
retention gained by training at k (pts)
40
30
20
10
0
2
4
8
16
36
fragments per design, k

Figure S1: **Training at k, against the control dropped from Fig. 2.** [PLACEHOLDER — the prose is yours.] **(a)** Retention against the number of fragments per design, one facet per backbone. Blue: a policy trained at that k and recombined at the same k. Grey: a single policy trained at $k = 4$ and re-cut at every k, with no training at the finer granularity. Every *trained* cell ran a matched 200 steps, so training length cannot covary with k; the control needs no training at all, which is what makes it the right counterfactual. **(b)** The difference between the two, which widens monotonically with k: +8.3 points at $k = 2$, +18.0 at $k = 8$, +27.9 at $k = 16$ and +32.2 at $k = 36$. Cutting a finished design finer loses recombinants faster than training a policy to be cut that finely does, and the finer the cut the more training at that granularity is worth. Caveats: every cell is single-seeded against a documented 21-point run-to-run swing; M0255 at $k = 36$ and $k = 64$ was still climbing at step 200, so those are lower bounds.